

Route Paramaters

In the last lesson, we looked at basic routing. In this lesson, we will look at route parameters. Route parameters are used to capture values from the URI. For example, you may have a url like this:

```
http://localhost:8000/user/1
```

In this case, the **1** is a route parameter. This is typically the ID of the resource, in this case, the user.

Open your `routes/web.php` file. You can capture params in your route definition like this:

```
Route::get('/post/{id}', function (string $id) {  
    return 'Post '.$id;  
});
```

In this case, the route parameter is `{id}`. The value of the route parameter is passed to the closure as an argument. In this case, the argument is `$id`. The closure then returns the string `Post` followed by the value of the route parameter.

Now visit `http://localhost:8000/post/1` and you will see the string `Post 1`.

Multiple Route Parameters

You can also have multiple route parameters:

```
Route::get('/posts/{post}/comments/{comment}', function (string $postId, string  
$commentId) {  
    return 'Post '.$postId.' Comment '.$commentId;  
});
```

In this case, we have two route parameters, `{post}` and `{comment}`. The values of these route parameters are passed to the closure as arguments. The closure then returns the string `Post` followed by the value of the `post` route parameter, followed by `Comment` followed by the value of the `comment` route parameter.

Now visit `http://localhost:8000/posts/1/comments/100` and you will see the string `Post 1 Comment 100`.

Optional Route Parameters

You can also have optional route parameters. For example:

```
Route::get('/user/{name?}', function ($name = null) {  
    return $name ? 'User '.$name : 'No user specified';  
});
```

```
});
```

Visit <http://localhost:8000/user> and you will see the string `No user specified`. Visit <http://localhost:8000/user/john> and you will see the string `User john`.

Route Parameter Constraints

You can also constrain route parameters. For example, you may want a route parameter to only accept numbers. You can do this like so:

```
Route::get('/user/{id}', function ($id) {  
    return 'User '.$id;  
})->where('id', '[0-9]+');
```

Now if you visit <http://localhost:8000/user/john>, you will get a 404 error. But if you visit <http://localhost:8000/user/1>, you will see the string `User 1`.

In addition to regular expressions, you can also use the following shortcuts:

- `where('id', '[0-9]+')` is the same as `whereNumber('id')`
- `where('id', '[a-zA-Z]+')` is the same as `whereAlpha('id')`
- `where('id', '[a-zA-Z0-9]+')` is the same as `whereAlphaNumeric('id')`

For example:

```
Route::get('/user/{id}', function ($id) {  
    return 'User '.$id;  
})->whereNumber('id');
```

or

```
Route::get('/user/{id}/{name}', function (string $id, string $name) {  
    return 'User ' . $id . ' ' . $name;  
})->whereNumber('id')->whereAlpha('name');
```

Now if you visit <http://localhost:8000/user/john>, you will get a 404 error. But if you visit <http://localhost:8000/user/1>, you will see the string `User 1`.

Global Constraints

You can also define global constraints in the `App\Providers\AppServiceProvider` class. Open that file and add the following import:

```
use Illuminate\Support\Facades\Route;
```

Then add the following code to the `boot` method:

```
public function boot(): void
{
    Route::pattern('id', '[0-9]+');
}
```

Now use the following route in the `routes/web.php` file:

```
Route::get('/user/{id}', function ($id) {
    return 'User ' . $id;
});
```

Now if you visit `http://localhost:8000/user/john`, you will get a 404 error. But if you visit `http://localhost:8000/user/1`, you will see the string `User 1`. Any route parameter named `id` will now be constrained to only accept numbers.